

Conditional Jacobian Aggregation

Optimizing GPU Track Finding through Algorithmic Refinement

Hao-Chun, Liang

GPU Optimization for ACTS/traccc

January 2026

Advisor: Prof. Bo-Cheng, Lai

Outline

- 1 Problem Context
- 2 Implementation
- 3 Results
- 4 Conclusion

Register Pressure in GPU Track Finding

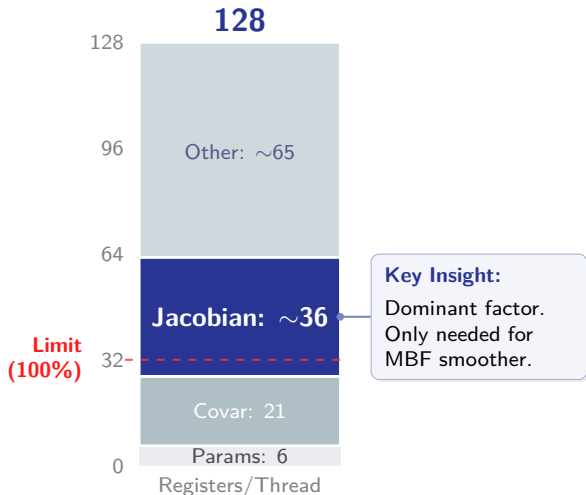
The Problem

- propagate_to_next_surface kernel
- Only **10-25% GPU occupancy**
- Root cause: High register usage
- V100 needs ≤ 32 regs for 100% occupancy

Impact

- ✗ SM underutilization
- ✗ Poor latency hiding
- ✗ Reduced throughput

Register Budget Breakdown



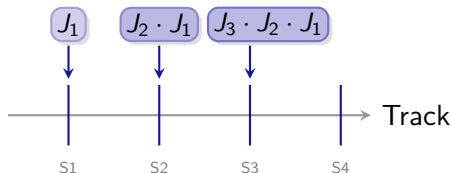
Opportunity: Conditional Jacobian

When is Jacobian Needed?

- Multi-Branch Fit (MBF) smoother requires accumulated Jacobian
- Used for backward smoothing pass
- **Scenario:** When MBF disabled $\rightarrow$ Jacobian computed but **never used**

Current Behavior (parameter_transporter)

- 1 Compute full 6×6 Jacobian at each surface
- 2 Multiply: $J_{acc} = J_{new} \times J_{acc}$
- 3 Store accumulated Jacobian to global memory



Savings (per surface)

- ~ 216 FLOPs (matrix mult)
- ~ 288 bytes memory I/O

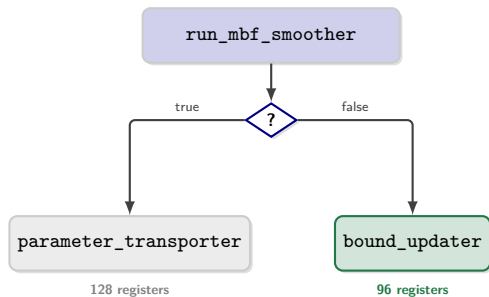
Solution: Two Actor Chains

Original: `ckf_actor_chain_t`

- 1 `pathlimit_aborter`
- 2 `parameter_transporter` $\leftarrow$ Jacobian ptr
- 3 `interaction_register`
- 4 ...

New: `ckf_actor_chain_no_mbf_t`

- 1 `pathlimit_aborter`
- 2 `bound_updater` $\leftarrow$ **Empty state!**
- 3 `interaction_register`
- 4 ...

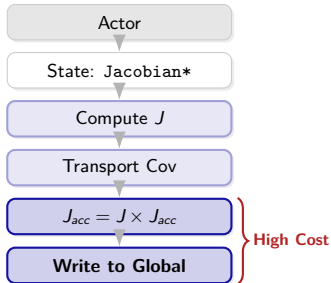


Compile-Time Dispatch: if constexpr selects actor chain based on propagator type.

Key Code: Actor State Comparison

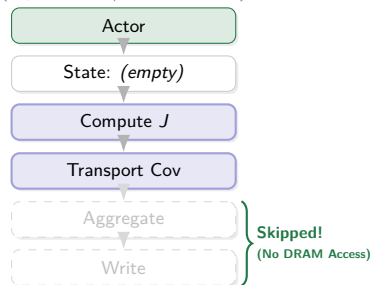
parameter_transporter

(Original / MBF=true)



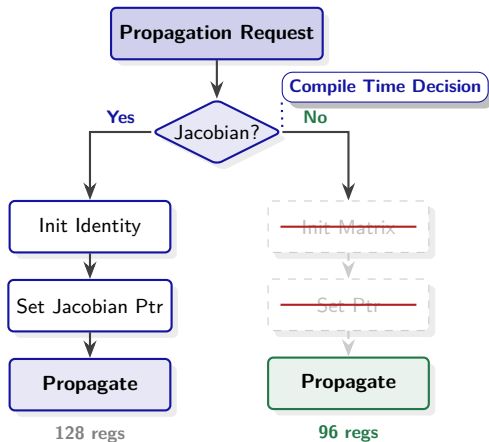
bound_updater

(Optimized / MBF=false)



Key Difference: Empty state → **Eliminates** 6×6 matrix multiplication & Global Memory Write

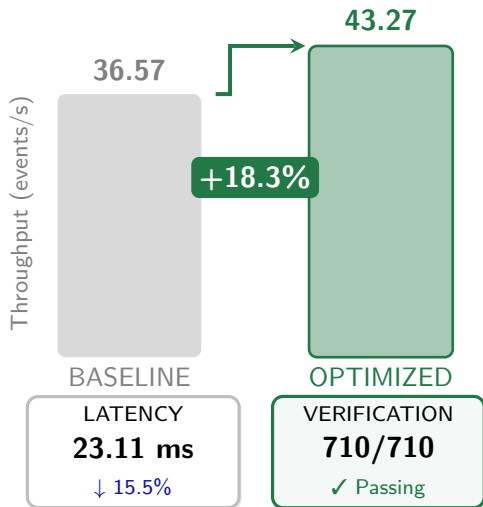
Kernel Dispatch Logic



18 Kernel Specializations

3 detector types × 3 B-field configs × 2 MBF variants

Benchmark Results



Config: V100-SXM2 (8 threads)

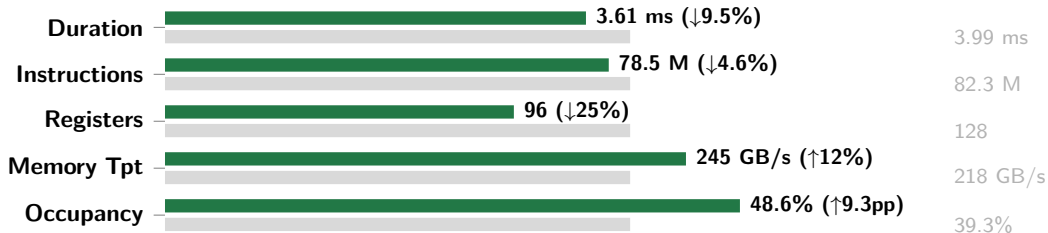
Dataset: ttbar_mu200

Test Condition

Apples-to-apples comparison:
Both with MBF=false

NCU Profiling: Register Reduction Confirmed

Kernel: *propagate_to_next_surface* | Hardware: RTX 2080 Ti (sm_75)



Dual Optimization Mechanism

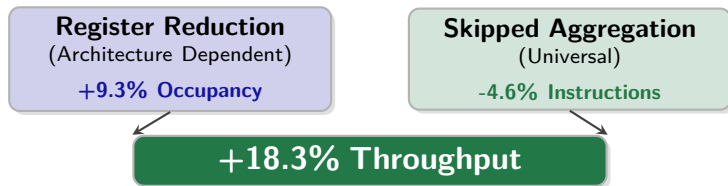
1. Register Reduction

Theoretical Occ.: 50% → **62.5%**
(Achieved Occ.: 39.3% → 48.6%)

2. Instruction Reduction

Less compute work
(82.3M → 78.5M)

Optimization Mechanism Summary



Per Track Savings: $\sim 3,240$ FLOPs and $\sim 4,320$ bytes memory I/O

Conclusion

Paper Analysis (Theoretical)

- Registers: -36
- Occupancy: +10-25%
- Throughput: +5-15%

Real Hardware (sm_75)

- Registers: -32 ✓
- Occupancy: +9.3% ✓
- **Throughput: +18.3% ✓**

Caveat

Optimization only works when **MBF smoother is disabled**. Trade-off: No backward smoothing.

Achievements

- ✓ **+18.3%** throughput
- ✓ **-25%** register usage (sm_75)
- ✓ **710/710** tests passing

Implementation

- `bound_updater` actor
- 18 kernel specializations
- Compile-time dispatch

Mechanism

- 1 Skip 6×6 matrix multiplication
- 2 Reduce global memory traffic
- 3 Architecture-dependent register savings

Key Files

- `bound_updater.hpp`
- `propagate_to_next_surface.ipp`

Thank you!

+18.3%
Throughput

-25%
Registers

710/710
Tests Passed

github.com/acts-project/traccc

Q1: Why does Jacobian computation still happen?

- `get_full_jacobian()` is needed for covariance transport. We only skip the *aggregation*.

Q2: Why not runtime dispatch?

- `if constexpr` enables dead code elimination and specific register allocation.

Q3: Why different results on sm_70 vs sm_75?

- Compiler optimizations, instruction sets (tensor cores), and register allocation heuristics vary by architecture.